



(21) 申请号 202511559341.7

(22) 申请日 2025.10.29

(65) 同一申请的已公布的文献号

申请公布号 CN 121029434 A

(43) 申请公布日 2025.11.28

(73) 专利权人 中科南京信息高铁研究院

地址 211100 江苏省南京市江宁区创研路

266号麒麟人工智能产业园8号楼1-2

层

(72) 发明人 张宗帅 张思雨 邹文浩 田霖

武广智

(74) 专利代理机构 南京知识律师事务所 32207

专利代理师 王克娟

(51) Int.Cl.

G06F 9/50 (2006.01)

G06F 9/48 (2006.01)

G06N 3/042 (2023.01)

G06N 3/092 (2023.01)

G06N 3/048 (2023.01)

(56) 对比文件

CN 117596301 A, 2024.02.23

CN 120371476 A, 2025.07.25

审查员 张春艳

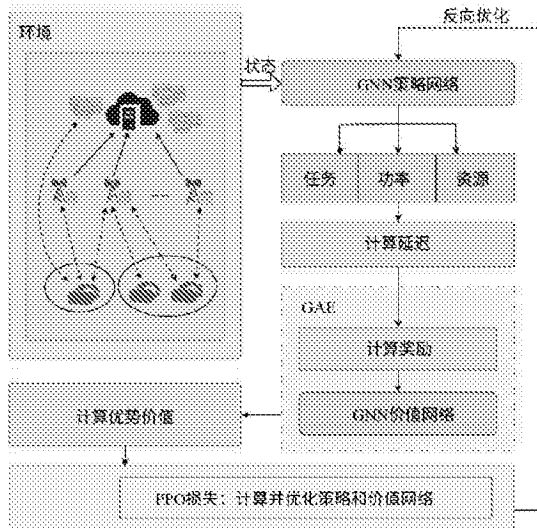
权利要求书2页 说明书8页 附图4页

(54) 发明名称

一种基于动态云边协同的任务调度方法

(57) 摘要

本发明属于移动边缘计算技术领域,公开了一种基于动态云边协同的任务调度方法。本发明通过链路级建模与图神经网络特征提取,实现了对不同数量终端和服务器的自适应扩展;本发明结合近端策略优化算法的裁剪更新和广义优势估计方法,有效避免了训练中的不稳定性和过估计问题。本发明既能充分利用云端弹性算力和边缘低时延特性,又能适配动态变化的网络规模和终端需求,可实现任务的灵活卸载和动态资源分配,在动态环境下有效减少任务处理延迟,提高资源利用率,并在终端数量和服务器规模波动的情况下保持良好的扩展性和稳定性,解决了现有移动边缘计算在虚拟现实视频服务中存在的计算资源瓶颈与任务调度僵化问题,显著改善了用户的交互体验。



1. 一种基于动态云边协同的任务调度方法,其特征在于,包括以下步骤:

步骤1:系统建模与异构图构建:

步骤1.1:服务器包括云服务器 V_c 、边缘服务器 V_s ,建立包括云服务器 V_c 、边缘服务器 V_s 和终端 V_u 的云-边-端系统模型,在云边协同架构下,将云服务器节点、边缘服务器节点和终端节点建模为异构图,异构图 $G=(V_u, V_s, V_c, E)$,其中 E 表示所有可能的终端-服务器连接;

终端节点特征包括任务数据量 d_i ,边缘服务器节点特征包括计算能力 f_j ,终端节点-服务器节点间的链路特征包括信道增益 $h_{i,j}$;

步骤1.2:将每个终端任务按比例拆分为若干个子任务,分别卸载至不同的服务器,建立传输约束与计算资源约束;

步骤1.3:建立任务总时延模型,包括传输时延与计算时延,并存入初始调度表;

步骤2:基于图神经网络GNN生成调度策略:

步骤2.1:异构图特征提取:采用图神经网络GNN对终端节点特征、边缘服务器节点特征、终端节点-服务器节点间的链路特征进行聚合与更新,提取任务分配、功率分配、资源分配的关系;

步骤2.2:调度策略矩阵输出:通过图神经网络GNN前向输出,生成三类调度策略矩阵,包括任务分配矩阵 A_{task} 、功率分配矩阵 A_{power} 和资源分配矩阵 A_{comp} ,所述任务分配矩阵 A_{task} 包括终端与服务器之间的卸载比例,所述功率分配矩阵 A_{power} 包括链路传输功率分配,所述资源分配矩阵 A_{comp} 包括服务器计算资源分配比例;

步骤2.3:奖励信号定义:将总时延作为优化目标进行最小化操作,归一化处理后得到奖励函数;

步骤3:利用近端策略优化算法PPO的调度策略更新:

步骤3.1:将调度问题建模为马尔科夫决策过程MDP,状态 s 包括任务数据量、信道增益、计算能力,调度策略矩阵 a 为三类调度策略矩阵,奖励函数 r_e 为归一化时延函数;

步骤3.2:采用目标函数 $L_{clip} = \min(r_t \times A_t, \text{clip}(r_t, 1-\theta, 1+\theta) \times A_t)$ 实现调度策略的稳定迭代更新,其中, r_t 代表新调度策略和旧调度策略在同一个状态上的概率之比, A_t 代表优势函数, θ 代表裁剪参数,裁剪损失函数 $\text{clip}(r_t, 1-\theta, 1+\theta)$ 代表把 r_t 限制在 $[1-\theta, 1+\theta]$ 的范围内, $\min()$ 代表取最小值操作;

步骤3.3:通过广义优势估计方法GAE计算优势值,避免调度策略崩溃并提升收敛速度;

步骤3.4:在每次迭代中,图神经网络GNN生成的调度策略矩阵与步骤2.3奖励信号共同参与优化,最终得到动态调整策略网络;

步骤4:动态云边协同执行:

步骤4.1:当边缘服务器资源充足时,任务优先在本地边缘服务器执行;

步骤4.2:当边缘服务器负载过高时,部分子任务可卸载至云端服务器;

步骤4.3:当存在多个邻居边缘服务器时,任务可通过边缘服务器-边缘服务器协同完成分配,以降低单点过载风险。

2. 根据权利要求1所述的基于动态云边协同的任务调度方法,其特征在于,步骤1.2中,以链路 (i, j) 表示终端 i 与 j 之间的连接,定义任务分配比例为 $x_{i,j}$,传输功率为 $p_{i,j}$,分配给

服务器的计算资源为 $f_{i,j}$,则传输约束需满足 $\sum_{j=1}^M x_{i,j} + x_{i,cloud} = 1$ $x_{i,j} \geq 0$,其中, Σ 代表求和操作, M 代表终端的数量, $x_{i,cloud}$ 代表终端 i 与云服务器之间的任务分配比例;

计算资源约束需满足 $\sum_{j=1}^M f_{i,j} + f_{i,j}^c \leq f_i$ 其中, $f_{i,j}$ 代表分配给服务器的计算资源, $f_{i,j}^c$ 代表云计算资源, f_i 代表服务器总资源,保证服务器总资源不超过其最大计算能力。

3.根据权利要求1所述的基于动态云边协同的任务调度方法,其特征在于,步骤1.3建立任务总时延模型,包括传输时延与计算时延,并存入初始调度表,包括以下步骤:

步骤1.3.1:将传输时延表示为 $T_{i,j}^{tran} = \frac{x_{i,j}d_i}{r_{i,j}}$ 其中,

$r_{i,j} = b \log_2 \left(1 + \frac{p_{i,j}h_{i,j}}{2 \sum_{k \neq i} p_{k,j}h_{k,j} + \sigma^2} \right)$ 以链路 (i,j) 表示终端 i 与 j 之间的连接, $x_{i,j}$ 为终端 i 与 j 之间的任务分配比例, d_i 为任务数据量, $r_{i,j}$ 为终端 i 与 j 之间的链路传输速率, b 为带宽, $p_{i,j}$ 为终端 i 与 j 之间的传输功率, σ^2 为噪声功率, $h_{i,j}$ 为终端 i 与 j 之间的信道增益, $p_{k,j}$ 为终端 k 与 j 之间的传输功率, $h_{k,j}$ 为终端 k 与 j 之间的信道增益, N 为终端的数量;

步骤1.3.2:将计算时延表示为 $T_{i,j}^{com} = \frac{x_{i,j}d_i c}{f_{i,j}}$ 其中, c 为单位数据所需计算开销常数, $f_{i,j}$ 为分配给服务器的计算资源;

步骤1.3.3:总时延表示为 $T_{i,j} = T_{i,j}^{tran} + T_{i,j}^{com}$,总时延的值作为后续调度优化的核心目标函数,同时存入初始调度表。

4.根据权利要求1所述的基于动态云边协同的任务调度方法,其特征在于,步骤2.3中的奖励函数表示为 $r_e = -\frac{T_{i,j}^{max} - T_{smo}}{T_{i,j}^{max} - T_{i,j}^{min}}$ 奖励值在区间 $[0,1]$,其中,

$$T_{smo} = (1-s_f) \times T_{i,j} + s_f \times T_{i,j}$$

$$T_{i,j}^{min} = (1-a') \times T_{i,j}^{min} + a' \times C_{i,j}^{min}$$

$$T_{i,j}^{max} = (1-a') \times T_{i,j}^{max} + a' \times C_{i,j}^{max}$$

T_{smo} 为平滑处理后的时延, $T_{i,j}^{max}$ 、

$T_{i,j}^{min}$ 分别表示时间的最大值和最小值,通过指数滑动平均动态更新, s_f 的值为0.2,为时间平滑因子, a' 的值为0.1,为指数滑动平均动态更新系数, $C_{i,j}^{min}$ 和 $C_{i,j}^{max}$ 分别表示在时间 $T_{i,j}$ 的动态更新的任务处理延迟最小值和最大值。

一种基于动态云边协同的任务调度方法

技术领域

[0001] 本发明主要涉及移动边缘计算技术领域,特别是涉及一种基于动态云边协同的任务调度方法。

背景技术

[0002] 近年来,随着虚拟现实(VR)技术的快速发展,全球VR设备出货量持续增长。根据国际数据公司(IDC)的统计,2024年第三季度VR设备出货量同比增长12.8%。与此同时,越来越多的终端用户使用分辨率达到4K甚至8K、刷新率高达120Hz的头戴显示设备。这类高性能终端虽然显著提升了沉浸式体验,但其对计算与网络资源的需求也呈现指数级增长。

[0003] 移动边缘计算(Mobile Edge Computing, MEC)技术通过在网络边缘部署计算资源,能够缩短任务传输路径,降低时延,并满足VR、AR、云游戏等实时性和带宽敏感应用的需求。然而,由于VR应用涉及高清视频编解码、实时场景渲染、用户交互同步等复杂计算任务,单纯依赖边缘服务器往往面临算力不足的问题,尤其在高并发、多用户场景下更易出现资源拥塞和服务质量下降。

[0004] 现有的任务调度方法大多基于静态假设,例如假定终端设备数量和服务器资源固定不变。这种方式在实际动态网络环境下存在明显不足:当终端数量突然增加或网络链路条件波动时,静态调度易导致任务队列溢出、平均处理时延急剧上升,甚至引发用户严重的眩晕和不适,严重影响VR业务体验。此外,静态模型缺乏对资源动态变化的适应性,难以在不同规模的边缘-云协同系统中推广应用。

[0005] 已有技术尝试结合深度强化学习(DRL)、图神经网络(GNN)等智能优化方法,提升边缘-云任务调度的灵活性。传统的智能优化方法,如模拟退火和遗传算法,不适合具有毫秒级延迟灵敏度的系统。由于缺乏支持监督学习模型训练所需的历史数据,具有环境交互能力的强化学习(RL)方法已成为一种理想的替代方案。

[0006] 强化学习(RL)是一种机器学习方法,通过与环境的互动来学习决策策略。其核心原理是,智能体在给定状态下选择一个动作,环境根据该动作做出新的状态和相应的奖励响应。智能体根据这种反馈迭代地调整策略,以最大化长期累积奖励。在强化学习(RL)算法中,近端策略优化(PPO)因其稳定性和有效性而得到了广泛的应用。近端策略优化(PPO)通过限制每次迭代期间策略更新的大小来优化策略网络,从而保持训练稳定性并有效克服传统Deep Q-Network(DQN)算法中与离散动作空间相关的维数诅咒。近端策略优化(PPO)中的重要性采样机制显著提高了采样效率,这在需要对毫秒级VR交互进行实时响应的云边缘协作环境中尤为重要。

[0007] 当任务涉及具有复杂关系依赖关系的图结构数据时,多层感知机(MLP)通常无法捕获节点间依赖关系和拓扑结构。在这种情况下,图神经网络(GNN)提供了明显的优势。通过利用消息传递机制,图神经网络(GNN)显式地对节点之间的连接进行建模,并提取结构信息特征表示。因此,图神经网络(GNN)在任务卸载和资源分配等场景中表现出优越的性能。

[0008] 目前,已有方法提出通过深度强化学习减少冷启动开销,或基于图神经网络优化

内容缓存。但这些方法大多仍停留在节点级特征建模层面,缺乏对链路级动态关系的细粒度刻画,难以充分应对复杂多变的VR场景。现有技术在资源调度静态化、系统扩展性不足、任务延迟过高等方面存在局限。因此,亟需一种能够在云与边缘之间进行动态协同的任务调度方法,该方法既能充分利用云端弹性算力和边缘低时延特性,又能适配动态变化的网络规模和终端需求,以保障VR视频服务的实时性和稳定性。

发明内容

[0009] 针对现有方法多基于静态假设,无法适应终端数量和服务器资源动态波动,在资源调度静态化、系统扩展性不足、任务延迟过高等方面存在局限,本发明提供一种基于动态云边协同的任务调度方法,旨在实现任务的灵活卸载和动态资源分配,保障VR视频服务的实时性和稳定性,解决现有移动边缘计算(MEC)在虚拟现实(VR)视频服务中存在的计算资源瓶颈与任务调度僵化问题,改善用户交互体验。

[0010] 为达到上述目的,本发明提供了一种基于动态云边协同的任务调度方法,包括以下步骤:

[0011] 步骤1:系统建模与异构图构建:

[0012] 步骤1.1:服务器包括云服务器 V_c 、边缘服务器 V_s ,建立包括云服务器 V_c 、边缘服务器 V_s 和终端 V_u 的云-边-端系统模型,在云边协同架构下,将云服务器节点、边缘服务器节点和终端节点建模为异构图,异构图 $G = (V_u, V_s, V_c, E)$,其中 E 表示所有可能的终端-服务器连接;

[0013] 终端节点特征包括任务数据量 d_i ,边缘服务器节点特征包括计算能力 f_j ,终端节点-服务器节点间的链路特征包括信道增益 $h_{i,j}$;

[0014] 步骤1.2:将每个终端任务按比例拆分为若干个子任务,分别卸载至不同的服务器,建立传输约束与计算资源约束;

[0015] 步骤1.3:建立任务总时延模型,包括传输时延与计算时延,并存入初始调度表;

[0016] 步骤2:基于图神经网络GNN生成调度策略:

[0017] 步骤2.1:异构图特征提取:采用图神经网络GNN对终端节点特征、边缘服务器节点特征、终端节点-服务器节点间的链路特征进行聚合与更新,提取任务分配、功率分配、资源分配的关系;

[0018] 步骤2.2:调度策略矩阵输出:通过图神经网络GNN前向输出,生成三类调度策略矩阵,包括任务分配矩阵 A_{task} 、功率分配矩阵 A_{power} 和资源分配矩阵 A_{comp} ,任务分配矩阵 A_{task} 包括终端与服务器之间的卸载比例,功率分配矩阵 A_{power} 包括链路传输功率分配,资源分配矩阵 A_{comp} 包括服务器计算资源分配比例;

[0019] 步骤2.3:奖励信号定义:将总时延作为优化目标进行最小化操作,归一化处理后得到奖励函数;

[0020] 步骤3:利用近端策略优化算法PPO的调度策略更新:

[0021] 步骤3.1:将调度问题建模为马尔科夫决策过程MDP,状态 s 包括任务数据量、信道增益、计算能力,调度策略矩阵 a 为三类调度策略矩阵,奖励函数 r_e 为归一化时延函数;

[0022] 步骤3.2:采用目标函数 $L_{clip} = \min(r_t \times A_t, \text{clip}(r_t, 1-\theta, 1+\theta) \times A_t)$ 实现调度策略的稳定迭代更新,其中, r_t 代表新调度策略和旧调度策略在同一个状态上的概率之比, A_t 代

表优势函数, θ 代表裁剪参数, 裁剪损失函数 $\text{clip}(r_t, 1-\theta, 1+\theta)$ 代表把 r_t 限制在 $[1-\theta, 1+\theta]$ 的范围内, $\min()$ 代表取最小值操作;

[0023] 步骤3.3: 通过广义优势估计方法GAE计算优势值, 避免调度策略崩溃并提升收敛速度;

[0024] 步骤3.4: 在每次迭代中, 图神经网络GNN生成的调度策略矩阵与步骤2.3奖励信号共同参与优化, 最终得到动态调整策略网络;

[0025] 步骤4: 动态云边协同执行:

[0026] 步骤4.1: 当边缘服务器资源充足时, 任务优先在本地边缘服务器执行;

[0027] 步骤4.2: 当边缘服务器负载过高时, 部分子任务可卸载至云端服务器;

[0028] 步骤4.3: 当存在多个邻居边缘服务器时, 任务可通过边缘服务器-边缘服务器协同完成分配, 以降低单点过载风险。

[0029] 进一步的, 步骤1.2中, 以链路 (i, j) 表示终端 i 与 j 之间的连接, 定义任务分配比例为 $x_{i,j}$, 传输功率为 $p_{i,j}$, 分配给服务器的计算资源为 $f_{i,j}$, 则传输约束需满足

$$\sum_{j=1}^M x_{i,j} + x_{i,\text{cloud}} = 1 \quad x_{i,j} \geq 0, \text{ 其中, } \Sigma \text{ 代表将求和操作, } M \text{ 代表终端的数量, } x_{i,\text{cloud}} \text{ 代表终端 } i \text{ 与云服务器之间的任务分配比例;}$$

[0030] 计算资源约束需满足 $\sum_{j=1}^M f_{i,j} + f_{i,j}^c \leq f_i$ 其中, $f_{i,j}$ 代表分配给服务器的计算资源, $f_{i,j}^c$ 代表云计算资源, f_i 代表服务器总资源, 保证服务器总资源不超过其最大计算能力。

资源, $f_{i,j}^c$ 代表云计算资源, f_i 代表服务器总资源, 保证服务器总资源不超过其最大计算能力。

[0031] 进一步的, 步骤1.3建立任务总时延模型, 包括传输时延与计算时延, 并存入初始调度表, 包括以下步骤:

[0032] 步骤1.3.1: 将传输时延表示为 $T_{i,j}^{\text{tran}} = \frac{x_{i,j} d_i}{r_{i,j}}$ 其中,

$$r_{i,j} = b \log_2 \left(1 + \frac{p_{i,j} h_{i,j}}{2 \sum_{k \neq i}^N p_{k,j} h_{k,j} + \sigma^2} \right), \text{ 以链路 } (i, j) \text{ 表示终端 } i \text{ 与 } j \text{ 之间的连接, } x_{i,j}$$

为终端 i 与 j 之间的任务分配比例, d_i 为任务数据量, $r_{i,j}$ 为终端 i 与 j 之间的链路传输速率, b 为带宽, $p_{i,j}$ 为终端 i 与 j 之间的传输功率, σ^2 为噪声功率, $h_{i,j}$ 为终端 i 与 j 之间的信道增益, $p_{k,j}$ 为终端 k 与 j 之间的传输功率, $h_{k,j}$ 为终端 k 与 j 之间的信道增益, N 为终端的数量;

[0033] 步骤1.3.2: 将计算时延表示为 $T_{i,j}^{\text{com}} = \frac{x_{i,j} d_i c}{f_{i,j}}$ 其中, c 为单位数据所需计算开销常数, $f_{i,j}$ 为分配给服务器的计算资源;

[0034] 步骤1.3.3: 总时延表示为 $T_{i,j} = T_{i,j}^{\text{tran}} + T_{i,j}^{\text{com}}$, 总时延的值作为后续调度优

化的核心目标函数,同时存入初始调度表。

[0035] 进一步的,步骤2.3中的奖励函数表示为 $r_e = -\frac{T_{i,j}^{max} - T_{smo}}{T_{i,j}^{max} - T_{i,j}^{min}}$, 奖励值在区间[0,

1],其

[0036] $T_{smo} = (1-s_f) \times T_{i,j} + s_f \times T_{i,j}$

[0037] $T_{i,j}^{min} = (1-a') \times T_{i,j}^{min} + a' \times C_{i,j}^{min}$

[0038] 中, $T_{i,j}^{max} = (1-a') \times T_{i,j}^{max} + a' \times C_{i,j}^{max}$, T_{smo} 为平滑处理后的时延,

$T_{i,j}^{max}$ 、 $T_{i,j}^{min}$ 分别为时间的最大值和最小值,通过指数滑动平均动态更新, s_f 的值为0.2,为时间平滑因子, a' 的值为0.1,为指数滑动平均动态更新系数, $C_{i,j}^{min}$ 和 $C_{i,j}^{max}$ 分别表示在时间 $T_{i,j}$ 的动态更新的任务处理延迟最小值和最大值。

[0039] 有益效果:本发明提供一种基于动态云边协同的任务调度方法,通过链路级建模与图神经网络GNN特征提取,实现了对不同数量终端和服务器的自适应扩展,解决了传统多层感知机MLP方法输入输出维度固定的局限;结合近端策略优化算法PPO的裁剪更新和广义优势估计方法GAE,有效避免了训练中的不稳定性和过估计问题,保证了任务调度的高效与稳定;实验结果表明,本发明提出的动态边缘云协同调度方法DECS在任务处理延迟上较纯边缘计算和纯云计算架构分别减少了8.52%和20.54%,显著提升了VR业务中的实时交互体验。

附图说明

[0040] 图1是本发明实施例涉及的云边协同场景中的基本结构图;

[0041] 图2是本发明实施例涉及的云边协同场景中的可扩展的动态边缘云协同调度方法DECS的算法结构图;

[0042] 图3是本发明实施例涉及的不同模型下的奖励对比图;

[0043] 图4是本发明实施例涉及的不同服务器数量下不同模型的任务处理延迟示意图;

[0044] 图5是本发明实施例涉及的不同边缘服务器数量的任务处理延迟示意图;

[0045] 图6是本发明实施例涉及的不同终端数量的任务处理延迟示意图。

具体实施方式

[0046] 如图1至图6所示,为了解决现有边缘计算在虚拟现实(VR)任务调度中存在的资源分配僵化、无法适应动态终端规模和计算资源波动的问题,本发明提供一种基于动态云边协同的任务调度方法,动态边缘云协同调度方法(DECS)基于图神经网络(GNN)与近端策略优化(PPO)。该方法以云边协同架构为基础,结合强化学习策略更新机制,能够在任意数量的终端和边缘节点下实现任务的灵活卸载和资源优化分配,从而有效降低VR视频服务的任务处理时延,改善用户交互体验。

[0047] 实施例1:如图1所示,本发明的系统模型由终端用户、边缘服务器、云服务器组成,终端用户通过基站与边缘服务器相连,边缘服务器之间通过本地局域网实现协同,边缘服

务器通过回程链路与云中心相连。具体实施流程如图2所示。

[0048] 步骤1:系统建模与任务图构建

[0049] 在初始任务调度阶段,终端用户产生的计算任务尚未被分配,所有任务默认需要通过云中心处理。此时作为流程的开始,终端与服务器的任务分配关系尚未建立,必须首先构建系统模型与任务图,以便后续的动态调度优化,如图3所示。

[0050] 步骤1.1:节点与链路初始化,将终端记为 V_u ,边缘服务器 V_s ,云服务器记为 V_c 。对于终端节点,其特征为任务数据量 d_i ,对于边缘服务器节点,其特征为计算能力 f_j ,对于云服务器节点,设置计算能力为远高于边缘端,并设其传输链路延迟为边缘链路的两倍。链路特征包括信道增益 $h_{i,j}$,用于表示终端与服务器之间的信道质量。由此,构建异构图 $G=(V_u, V_s, V_c, E)$,其中 E 表示所有可能的终端—服务器连接。

[0051] 步骤1.2:将每个终端任务按比例拆分为若干个子任务,分别卸载至不同的服务器,建立传输约束与计算资源约束;

[0052] 以链路 (i, j) 表示终端 i 与 j 之间的连接,定义任务分配比例为 $x_{i,j}$,传输功率为

$$p_{i,j}, \text{分配给服务器的计算资源为 } f_{i,j}, \text{则传输约束需满足 } \sum_{j=1}^M x_{i,j} + x_{i,cloud} = 1, x_{i,j} \geq 0, \text{其中, } \Sigma \text{ 代表将求和操作, } M \text{ 代表终端的数量, } x_{i,cloud} \text{ 代表终端 } i \text{ 与云服务器之间的任务分配比例;}$$

[0053] 计算资源约束需满足 $\sum_{j=1}^M f_{i,j} + f_{i,j}^c \leq f_i$ 其中, $f_{i,j}$ 代表分配给服务器的计算

资源, $f_{i,j}^c$ 代表云计算资源, f_i 代表服务器总资源,保证服务器总资源不超过其最大计算能力。

[0054] 步骤1.3:建立任务总时延模型,包括传输时延与计算时延,并存入初始调度表;

[0055] 包括以下步骤:

[0056] 步骤1.3.1:将传输时延表示为 $T_{i,j}^{tran} = \frac{x_{i,j}d_i}{r_{i,j}}$ 其中,

$$r_{i,j} = b \log_2 \left(1 + \frac{p_{i,j}h_{i,j}}{2 \sum_{k \neq i}^N p_{k,j}h_{k,j} + \sigma^2} \right), \text{以链路 } (i, j) \text{ 表示终端 } i \text{ 与 } j \text{ 之间的连接, } x_{i,j} \text{ 为终端 } i \text{ 与 } j \text{ 之间的任务分配比例, } d_i \text{ 为任务数据量, } r_{i,j} \text{ 为终端 } i \text{ 与 } j \text{ 之间的链路传输速率, } b \text{ 为带宽, } p_{i,j} \text{ 为终端 } i \text{ 与 } j \text{ 之间的传输功率, } \sigma^2 \text{ 为噪声功率, } h_{i,j} \text{ 为终端 } i \text{ 与 } j \text{ 之间的信道增益, } p_{k,j} \text{ 为终端 } k \text{ 与 } j \text{ 之间的传输功率, } h_{k,j} \text{ 为终端 } k \text{ 与 } j \text{ 之间的信道增益, } N \text{ 为终端的数量;}$$

[0057] 步骤1.3.2:将计算时延表示为 $T_{i,j}^{com} = \frac{x_{i,j}d_i c}{f_{i,j}}$ 其中, c 为单位数据所需计算开销常数, $f_{i,j}$ 为分配给服务器的计算资源;

[0058] 步骤1.3.3:总时延表示为 $T_{i,j} = T_{i,j}^{tran} + T_{i,j}^{com}$,总时延的值作为后续调度优化的核心目标函数,同时存入初始调度表,用于后续GNN建模与PPO策略训练。通过步骤1的初始化,系统在建模阶段完成了终端节点、服务器节点及其链路关系的定义,建立了任务拆分比例、资源约束条件以及时延计算模型,为后续基于图神经网络的特征提取和调度优化奠定了基础。

[0059] 步骤2:基于图神经网络GNN生成调度策略:

[0060] 步骤2.1:异构图特征提取:采用图神经网络GNN对终端节点特征、边缘服务器节点特征、终端节点-服务器节点间的链路特征进行聚合与更新,提取任务分配、功率分配、资源分配的关系;

[0061] 步骤2.2:调度策略矩阵输出:通过图神经网络GNN前向输出,生成三类调度策略矩阵,包括任务分配矩阵 A_{task} 、功率分配矩阵 A_{power} 和资源分配矩阵 A_{comp} ,任务分配矩阵 A_{task} 包括终端与服务器之间的卸载比例,功率分配矩阵 A_{power} 包括链路传输功率分配,资源分配矩阵 A_{comp} 包括服务器计算资源分配比例;

[0062] 步骤2.3:奖励信号定义:将总时延作为优化目标进行最小化操作,归一化处理后得到奖励函数;

[0063] 为了指导调度优化,将时延最小化目标转化为奖励函数,表示为

$$r_e = -\frac{T_{i,j}^{max} - T_{sma}}{T_{i,j}^{max} - T_{i,j}^{min}},$$

$$T_{sma} = (1-s_f) \times T_{i,j} + s_f \times T_{i,j}$$

$$T_{i,j}^{min} = (1-a') \times T_{i,j}^{min} + a' \times C_{i,j}^{min}$$

$$[0066] \quad \text{其中, } T_{i,j}^{max} = (1-a') \times T_{i,j}^{max} + a' \times C_{i,j}^{max}, T_{sma} \text{ 为平滑处理后的时延,}$$

$T_{i,j}^{max}$ 、 $T_{i,j}^{min}$ 分别为时间的最大值和最小值,通过指数滑动平均动态更新, s_f 的值为0.2,

为时间平滑因子, a' 的值为0.1,为指数滑动平均动态更新系数, $C_{i,j}^{min}$ 和 $C_{i,j}^{max}$ 分别表示在时间 $T_{i,j}$ 的动态更新的任务处理延迟最小值和最大值。

[0067] 步骤3:利用近端策略优化算法PPO的调度策略更新:

[0068] 步骤3.1:将调度问题建模为马尔科夫决策过程MDP,状态 s 包括任务数据量、信道增益、计算能力,调度策略矩阵 a 为三类调度策略矩阵,奖励函数 r_e 为归一化时延函数;

[0069] 步骤3.2:调度策略更新:采用的目标函数表示为 $L_{clip} = \min(r_t \times A_t, \text{clip}(r_t, 1-\theta, 1+\theta) \times A_t)$ 实现调度策略的稳定迭代更新,其中, r_t 代表新调度策略和旧调度策略在同一个状态上的概率之比, A_t 代表优势函数, θ 代表裁剪参数,裁剪损失函数 $\text{clip}(r_t, 1-\theta, 1+\theta)$ 代表把 r_t 限制在 $[1-\theta, 1+\theta]$ 的范围内, $\min()$ 代表取最小值操作。目标函数结合优势函数与裁剪损失函数,实现调度策略的稳定迭代更新;

[0070] 步骤3.3:通过广义优势估计方法GAE计算优势值,表示为 $A_t = \delta_t + \gamma \lambda A_{t+1}$,其中, γ 为折扣因子, λ 为平衡偏差与方差的因子, δ_t 为时序差分误差,若 $\delta_t > 0$,说明当前动作比预期更好,若 $\delta_t < 0$,说明比预期更差,避免调度策略崩溃并提升收敛速度;

[0071] 步骤3.4:在每次迭代中,图神经网络GNN生成的调度策略矩阵与步骤2.3奖励信号

共同参与优化,最终得到动态调整策略网络;

[0072] 步骤4:动态云边协同执行:

[0073] 步骤4.1:当边缘服务器资源充足时,任务优先在本地边缘服务器执行;

[0074] 步骤4.2:当边缘服务器负载过高时,部分子任务可卸载至云端服务器;

[0075] 步骤4.3:当存在多个邻居边缘服务器时,任务可通过边缘服务器-边缘服务器协同完成分配,以降低单点过载风险。

[0076] 该过程确保了任务在动态环境下的高效分配与时延最小化。

[0077] 为了验证本发明的效果,采用本发明提出的动态边缘云协同调度方法(DECS)动态云边协同任务调度方法与传统的边缘计算方法和云计算方法进行对比。对比方法包括:

[0078] 1) 纯边缘计算架构:所有任务均由边缘服务器处理;

[0079] 2) 纯云计算架构:所有任务均卸载至云服务器处理;

[0080] 3) 传统MLP+DQN调度方法:采用固定输入输出维度的神经网络进行任务分配。

[0081] 实验场景中,设置终端数量为服务器数量的两倍,仿真对比了不同方法在训练收敛性、任务处理延迟以及资源利用率方面的表现。

[0082] 如图3所示,不同方法在训练迭代次数增加时的奖励值变化趋势。可以看出,传统MLP+DQN方法在初期震荡较大,收敛速度较慢,最终奖励值低于本发明提出的DECS方法。而DECS方法在300轮迭代后逐渐趋于稳定,奖励值逼近最优水平,说明本发明在调度策略学习方面更稳定。

[0083] 如图4和图5所示,不同方法在不同服务器规模下的任务处理时延对比。结果表明:纯边缘计算架构在服务器数量有限时,因单点计算能力不足,平均任务处理时延较高;纯云计算架构虽然计算能力强,但远程链路传输带来的额外延迟显著增加了总时延;本发明提出的DECS方法在云边协同架构下,能够自适应地调整卸载比例和资源分配,始终保持较低的处理时延。与纯边缘计算和纯云计算方法相比,平均任务处理时延分别降低了8.52%和20.54%。

[0084] 如图6所示,比较了不同方法在终端激增情况下的任务延迟变化趋势。当终端数量快速增加时,纯边缘计算方法的延迟呈线性增长,纯云计算方法则因链路负载增加而性能下降。而本发明的DECS方法通过动态调节云边卸载比例,使延迟增长曲线显著趋缓,在大规模场景下依然优于对比方法。

[0085] 如表1所示,CPU资源利用率的对比结果表明:在相同规模下,纯边缘计算因单机能力有限利用率过高,纯云计算则存在资源闲置,而本发明的DECS方法能够在云与边之间合理分配负载,使得边缘侧利用率降低至约68%,云侧利用率稳定在30%左右,从而有效避免了单点过载与资源浪费问题;

[0086] 表1 CPU资源利用率对比结果

	服务器	DECS边	DECS云	纯云	纯边
[0087]	2	84%	46%	89%	60%
	3	79%	42%	88%	58%
	4	76%	39%	86%	56%
	5	74%	37%	85%	55%
	6	72%	35%	84%	54%
	7	71%	33%	83%	53%
	8	70%	31%	82%	52%
	9	69%	30%	81%	51%
	10	68%	29%	80%	50%

[0088] 本发明提供一种基于动态云边协同的任务调度方法,资源分配比例从节点级特征转移到链路级特征,其中终端和服务端之间的每个链路被分配一个独立的特征表示,由于模型在链路级别处理特征,而不是在节点级别产生固定维度的输出,因此可通过增加链路数量自然地容纳额外的终端和服务端,而无需修改模型的结构或参数。本发明既能充分利用云端弹性算力和边缘低时延特性,又能适配动态变化的网络规模和终端需求,可实现任务的灵活卸载和动态资源分配,在动态环境下有效减少任务处理延迟,提高资源利用率,并在终端数量和服务端规模波动的情况下保持良好的扩展性和稳定性,解决了现有移动边缘计算(MEC)在虚拟现实(VR)视频服务中存在的计算资源瓶颈与任务调度僵化问题,显著改善了用户的交互体验。

[0089] 最后应说明的是:以上仅为本发明的优选实施例而已,并不用于限制本发明,尽管参照实施例对本发明进行了详细的说明,对于本领域的技术人员来说,其依然可以对前述各实施例所记载的技术方案进行修改,或者对其中部分技术特征进行等同替换,但是凡在本发明的精神和原则之内,所作的任何修改、等同替换、改进等,均应包含在本发明的保护范围之内。

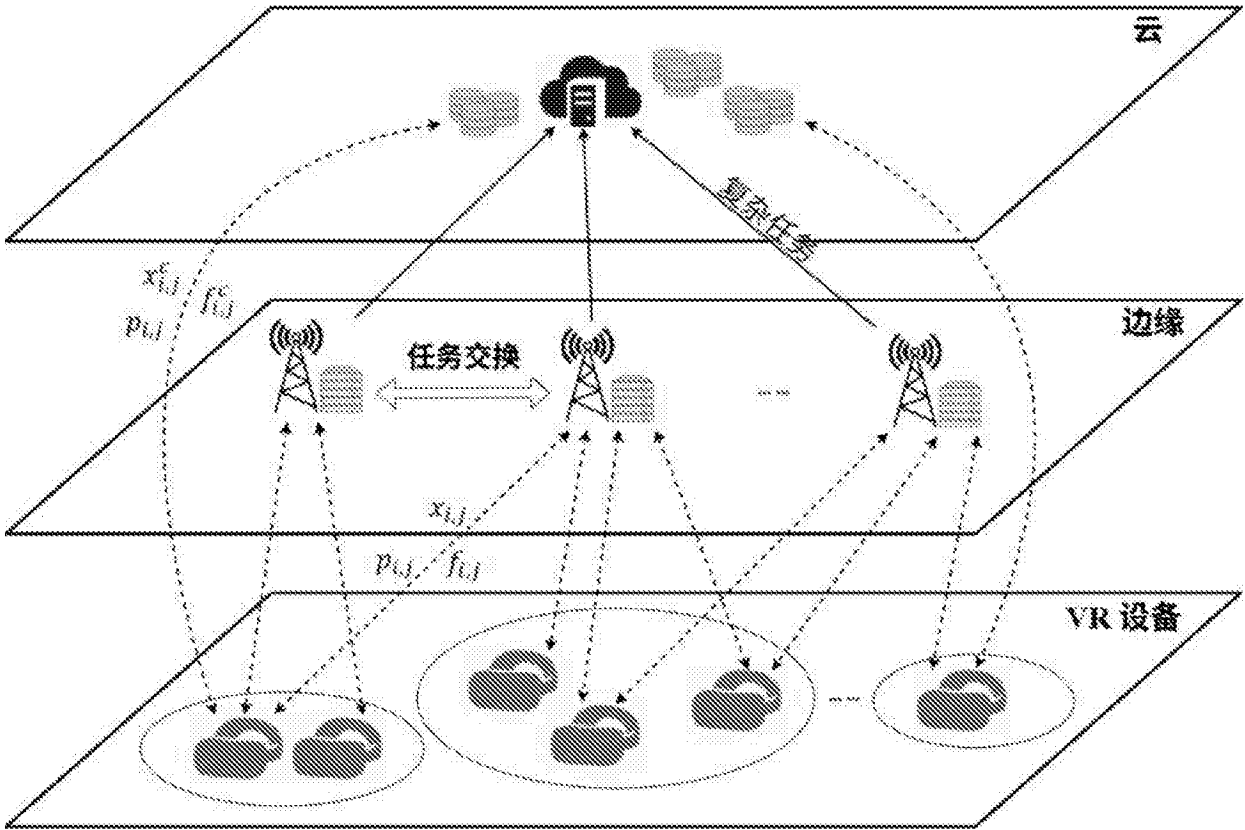


图 1

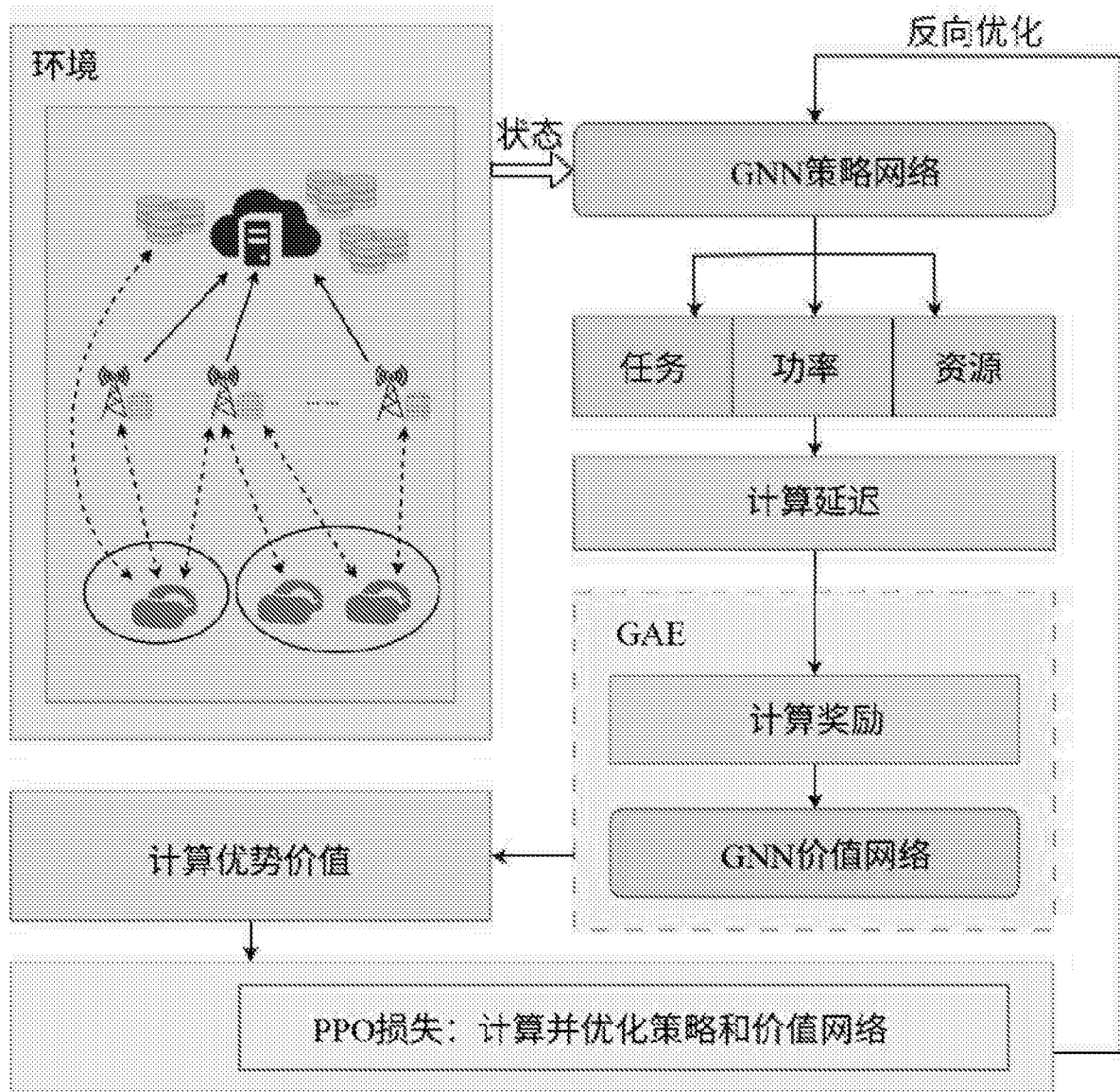


图 2

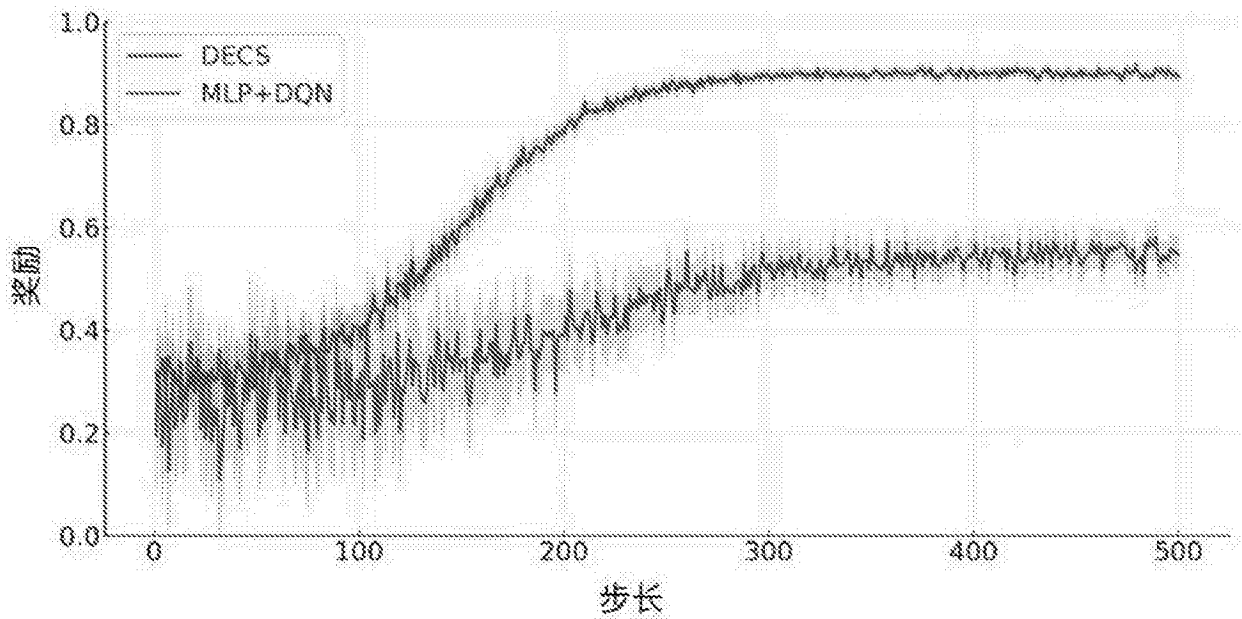


图 3

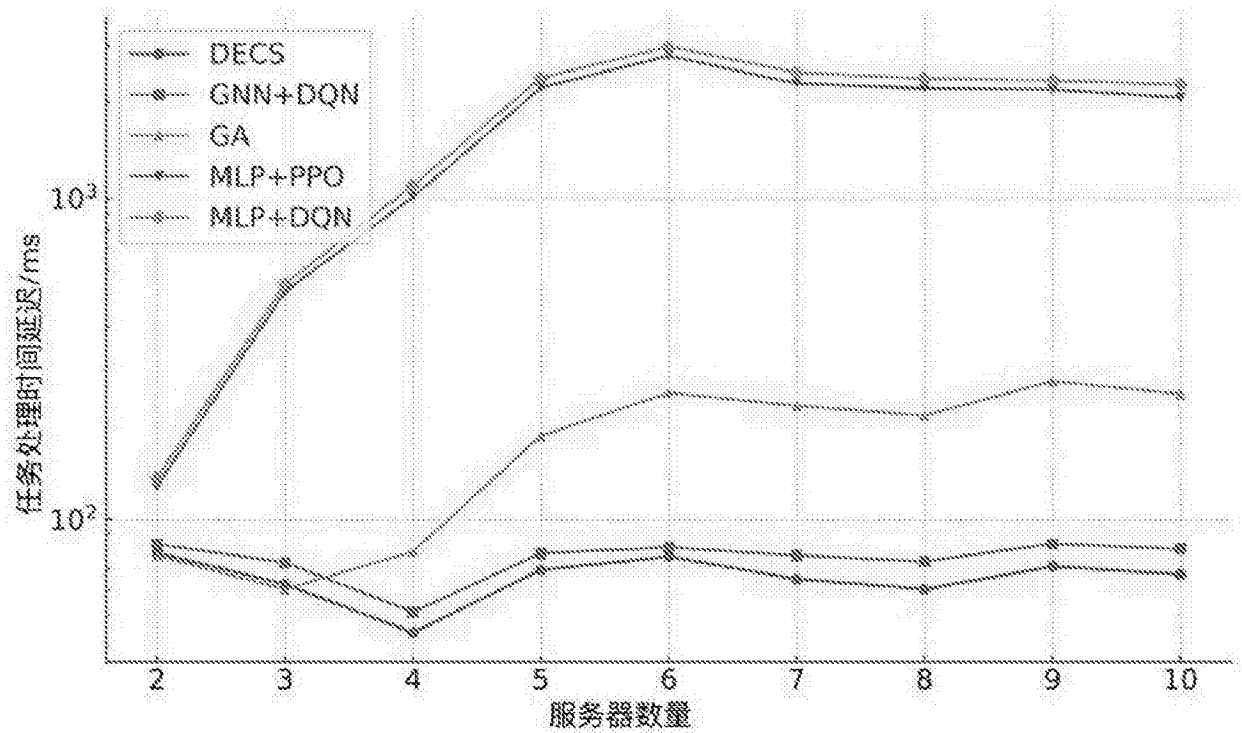


图 4

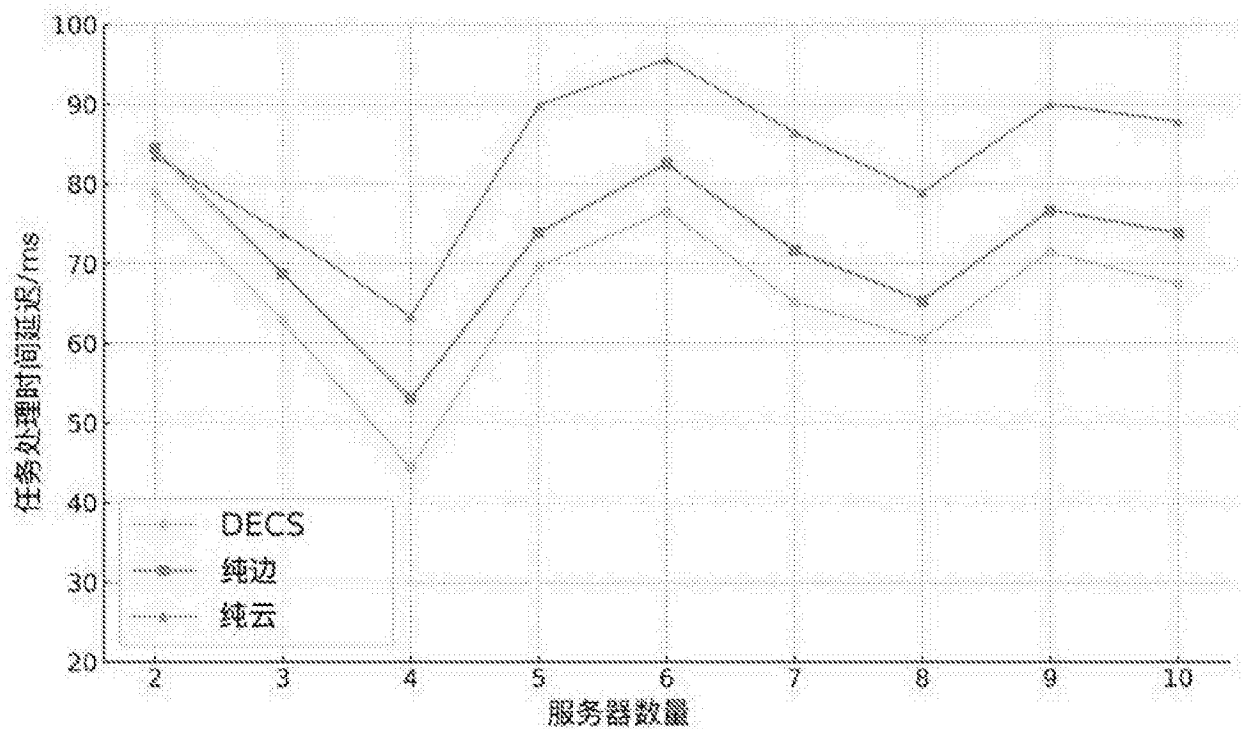


图 5

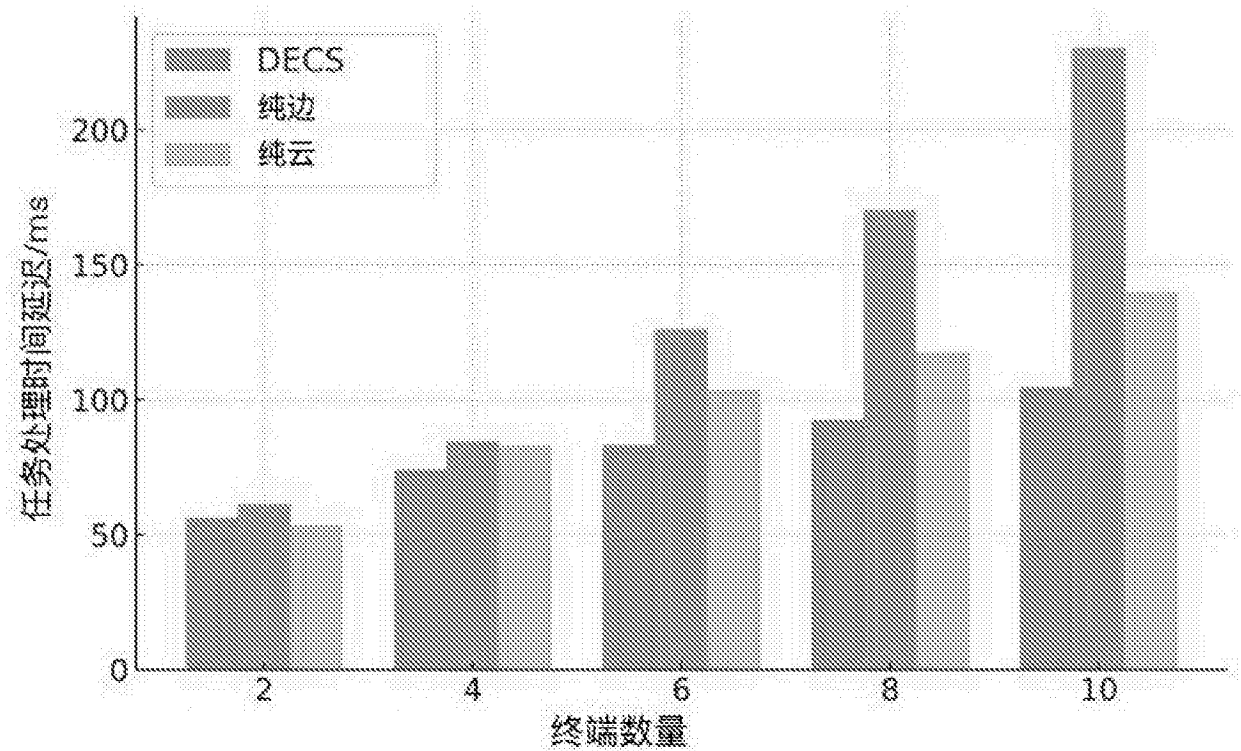


图 6